

ENTERPRISE PRODUCT DOSSIER

Confidential — prepared for client presentation

Shraddha Garden Resort

Complete Product Analysis, Feature Documentation, Architecture
Review, Product Audit & Presentation Playbook

"A sanctuary of celebration and stays"

PUBLIC SITE

ADMIN PORTAL

REST API

PAYMENTS

RBAC

Document type · Product & Technical Documentation + Investor/Client Walkthrough

Version · 1.0 | Date · 09 July 2026 | Status · For Review

Product · Shraddha Garden Resort — Coimbatore, Tamil Nadu, India

Prepared by · Product & Solution Architecture Practice

Table of Contents

<u>0 Executive Summary</u>	<u>03</u>
<u>1 Product Overview & Business Context</u>	<u>04</u>
<u>2 System Map, Routes & User Roles</u>	<u>05</u>
<u>3 Public Website — Screen-by-Screen Documentation</u>	<u>06</u>
<u>4 The Booking & Payment Engine</u>	<u>12</u>
<u>5 Admin Portal — Screen-by-Screen Documentation</u>	<u>14</u>
<u>6 Shared UI Component & Text Catalogue</u>	<u>22</u>
<u>7 Technical Architecture (Plain Language)</u>	<u>24</u>
<u>8 Functional Analysis — Input / Process / Output</u>	<u>26</u>
<u>9 Product Audit — Gaps & Missing Features</u>	<u>27</u>
<u>10 Negative Analysis — Risks (Brutally Honest)</u>	<u>29</u>
<u>11 Risk Matrix</u>	<u>30</u>
<u>12 Production Readiness Scorecard</u>	<u>31</u>
<u>13 Future Roadmap & Timeline</u>	<u>32</u>
<u>14 Business Benefits & ROI</u>	<u>34</u>
<u>15 100 Client Questions & Answers</u>	<u>35</u>
<u>16 Presentation Scripts (10 / 30 / 60 min)</u>	<u>39</u>
<u>17 Conclusion & Recommendation</u>	<u>42</u>

How to read this document. It is written for a non-technical decision-maker. Business meaning comes first in plain English; a short *"In technical terms"* note follows wherever it builds confidence. Nothing on the screen is assumed to be understood — every page, panel, button, label and message is named and explained.

Scope note: This dossier documents the product as implemented in the codebase (React public site, React admin portal, Express/Prisma/SQLite API). Where behaviour is inferred from the code rather than observed live, it is marked "inferred".

0 • Executive Summary

Shraddha Garden Resort is a complete, production-grade digital platform for a luxury garden resort in Coimbatore. It is not a brochure website — it is an **end-to-end hospitality operating system** with three connected parts working as one product.

1 • THE GUEST WEBSITE

A beautifully designed public site where guests discover the property, browse stays and amenities, explore dining and events, read offers, and **book and pay online** in a guided multi-step flow. Guests can create an account and view their bookings.

2 • THE ADMIN PORTAL

A secure staff back-office (/admin) to run the business day-to-day: bookings, availability & pricing, stays, marketing content, coupons, banners, reviews, enquiries, media, reports, users, payments and settings — with role-based permissions.

3 • THE API & DATA LAYER

A secure server that holds all data, enforces the rules (who can do what, valid booking states, correct prices), talks to the payment gateway, and sends automated emails. The website and admin both rely on it.

COMMERCIAL POSITION

Replaces a stack of separate tools (website builder + booking engine + CRM + CMS + spreadsheets) with **one owned platform**, removing per-booking commissions charged by travel portals (OTAs).

25+

screens / routes

4

user roles (RBAC)

40+

API endpoints

2

payment providers

16

content models

The one-paragraph pitch (read aloud)

“Shraddha Garden Resort now has its own online front door and its own control room. Guests can find the property, fall in love with it, and book a stay with real online payment — in minutes, from a phone. Behind the scenes, your team manages every booking, price, offer and enquiry from a single secure dashboard, with a full history of who changed what. Because you own the platform, every direct booking avoids the 15–25% commission that booking portals charge — and the same system is ready to grow into multiple properties.”

Overall verdict. The product is **feature-complete for a single-property direct-booking business** and demonstrably well-architected (RBAC, server-authoritative payments, audit

logging, state-machine-guarded bookings). The primary gaps before large-scale production are operational hardening — database scale, automated tests, monitoring and file-upload storage — detailed in Sections 9–12.

1 · Product Overview & Business Context

Product name: Shraddha Garden Resort · **Tagline:** “A sanctuary of celebration and stays.” ·

Location: Coimbatore, Tamil Nadu, India.

Contact identity carried across the product: email hello@shraddhagarden.com, phone +91 98765 43210, address “Coimbatore, Tamil Nadu, India”. Copyright line: “© 2024 Shraddha Garden Resort. Unforgettable Memories.”

What business problem it solves

BEFORE (TYPICAL RESORT)	AFTER (THIS PLATFORM)
Bookings by phone/WhatsApp; details lost in chats	Structured online booking with a unique code, stored centrally
Depends on OTAs (MakeMyTrip, Booking.com) that take 15–25% commission	Commission-free direct bookings on an owned channel
Prices and availability tracked in spreadsheets	Live availability + seasonal/dynamic rate rules + blocked dates
Marketing content changes need a web developer	Staff edit amenities, dining, events, offers, banners themselves
No record of who changed a booking or price	Full audit log of every admin change
Enquiries lost in an inbox	Leads inbox with status, assignee & convert-to-booking

Who uses it

PERSONA	WHERE	WHAT THEY DO
Guest / prospect	Public website	Browse, enquire, book, pay, create account, view own bookings, leave a review
Front Desk	Admin portal	View bookings, move booking status (check-in/out), handle enquiries & reviews
Manager	Admin portal	Everything Front Desk can, plus manage stays, pricing, content, coupons, banners, media, reports, settings, payments, refunds
Super Admin	Admin portal	Everything, plus create/manage staff & roles, edit payment secrets, delete stays, and view the Audit Log

Technology at a glance (plain language)

FRONT END (WHAT PEOPLE SEE)

React 18 + Vite + TypeScript, styled with Tailwind CSS. Smooth animations (Framer Motion). Charts (Recharts) and data tables (TanStack) in the admin. Fast, responsive on phone/tablet/desktop.

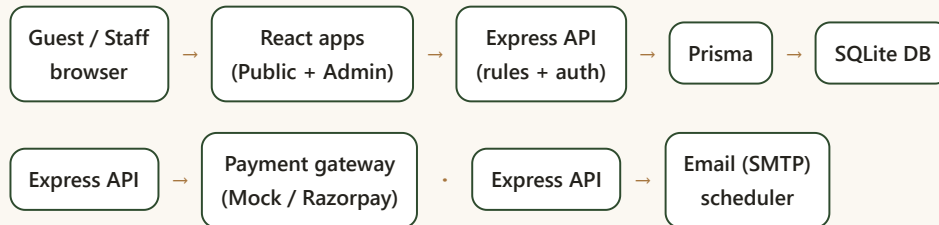
BACK END (THE ENGINE)

Node.js + Express server. Prisma toolkit over a SQLite database. Security via JWT tokens, bcrypt password hashing, and Zod input validation. Payments abstracted behind a provider layer (Mock & Razorpay).

Design system. A single set of brand tokens (colours cream/forest/terracotta/gold, Playfair Display + Inter typefaces, rounded “card” corners) is defined once and reused everywhere, so the public site and the admin portal feel like one premium brand.

2 • System Map, Routes & User Roles

2.1 High-level architecture



2.2 Public routes

PATH	SCREEN	ONE-LINE PURPOSE
/	Home	Hero, brand story, "Sixteen ways", steps, dining teaser, dismissible announcement bar
/about	About Us	Overlapping photos, stats, on-site list, team
/amenities	Amenities list	Filter tabs + 16-item grid + alternating features
/amenities/:slug	Amenity detail	Gallery, "What to expect", Quick Facts sidebar
/stays	Stays list	Sort dropdown + stay cards (data-driven)
/stays/:slug	Stay detail	Gallery, amenities, accordions, sticky booking widget
/dining	Dining	Cuisines, signature spreads, chef note
/events	Events	Hero, event types, gallery, steps
/offers	Offers/Packages	Promo packages (content type OFFER)
/reserve /enquiry	Reserve/Enquiry	Enquiry form → leads inbox
/book	Booking flow	Multi-step dates → availability → add-ons → details+coupon → review → confirmation
/signin	Sign in / up	Customer account (email + password)
/account	My Account	Profile + My Bookings
*	404 Not Found	On-theme fallback for unknown links

2.3 Admin routes & who can open them

PATH	SCREEN	MINIMUM ROLE
------	--------	--------------

/admin/login	Staff sign-in	Public
/admin	Dashboard (KPIs + charts)	Front Desk+
/admin/bookings	Bookings + detail drawer	Front Desk+
/admin/enquiries	Enquiries (leads inbox)	Front Desk+
/admin/reviews	Reviews moderation	Front Desk+
/admin/stays	Stays (delete = Super Admin)	Front Desk+ view / Manager+ edit
/admin/availability	Availability & Pricing	Manager+
/admin/amenities · dining · menu · events · offers	Content CMS	Manager+
/admin/coupons · banners · media	Coupons / Banners / Media	Manager+
/admin/reports	Reports + CSV export	Manager+
/admin/users	Users & Staff	Manager+ view / Super Admin manage
/admin/settings · email · payments	Settings / Email / Payments	Manager+ (secrets = Super Admin)
/admin/audit	Audit Log	Super Admin only

Security gate. Every admin screen sits behind `ProtectedRoute`: not-signed-in visitors are redirected to the login, and a role that lacks permission can never reach the page even by typing the URL.

03

Public Website

Screen-by-screen documentation of everything a guest sees — every page, panel, button, label, message, and state.

3 · Public Website — Screen by Screen

3.0 Global chrome — present on every public page

ANNOUNCEMENT BAR (TOP)

A thin strip above the header showing the current promotional message, driven by admin “Announcement” banners. It is **dismissible** — a close (×) button hides it for the visitor. *Business value*: push a seasonal message (“Monsoon offer — 20% off”) site-wide without a developer.

HEADER / NAVIGATION

Sticky header that is transparent over the hero image and turns solid on scroll. Left: **Shraddha Garden** wordmark (links home). Centre/right: nav links — **Home · About · Amenities · Stays · Dining · Events · Offers**. A primary **“Reserve”/“Book”** call-to-action. On mobile, an animated hamburger opens a full menu with body-scroll lock. *Every label* here maps to a route in §2.2.

FOOTER

Brand block with the tagline “A sanctuary of celebration and stays, blending modern luxury with natural serenity.”, quick links (mirroring the nav), contact details (email, phone, address), and the copyright line. Present on every public page as the closing landmark.

UNIVERSAL STATES

Data-driven pages (Stays, Amenities, Dining, Events, Offers) each render three explicit states via a shared `PublicState` helper: **LOADING** a pulsing placeholder, **EMPTY** a friendly “nothing here yet” message, and **ERROR** a retry message if the API is unreachable. A `ScrollToTop` behaviour resets scroll on every navigation.

3.1 Home /

Purpose: the emotional first impression and the primary funnel into booking. **Who**: every first-time visitor. **Why it exists**: convert curiosity into an enquiry or booking.

COMPONENT	CONTENT / LABELS	BUSINESS VALUE
Hero	Full-bleed image, resort name, tagline, primary CTA	Premium first impression; immediate route to book
Tradition band	Brand narrative copy, eyebrow “Discover”	Establishes the “celebration + stays” positioning
“Sixteen ways” grid	Showcase of the 16 amenities	Communicates breadth of the property at a glance
Steps band	01 Call & reserve · 02 Plan the day · 03 Celebrate (with descriptions)	Reduces friction — shows how easy booking an event is
Dining teaser	Eyebrow “Celebrate”, dining preview, CTA	Cross-sells the F&B experience
Stats / social proof	Numeric highlights (StatsRow)	Builds trust with figures

3.2 About Us /about

Purpose: tell the property's story and introduce the people. **Components:** overlapping hero photos, a stats row, an "on-site" list of what the estate offers, and a **team section** — Eloise Vance (Estate Director), Marcus Hume (Head of Celebrations), Sophia Lin (Guest Experience Lead), each on a TeamCard with role and photo. *Value:* humanises a high-consideration purchase (weddings/stays) and builds confidence.

3.3 Amenities list /amenities

Purpose: browse everything the property offers. **Components:** category **filter tabs**, a **16-item grid** of amenity cards, and alternating feature blocks. The 16 amenities documented in data are: Swimming Pool, Water Fall, Water Sprinklers, Water Snow, Water Fountain, Traditional Pump Set, Theatre House, Tower House, Wood House, Jividha, Kudil, Chillax Bay, Garden, Coconut Farm, Shuttle Court, and Camp Fire with DJ. Each card links to its detail page. *Value:* a scannable "menu of experiences" that drives interest and time-on-site.

3.4 Amenity detail /amenities/:slug

Purpose: deep-dive on one amenity. **Components:** a **gallery**, a "What to expect" narrative, and a **Quick Facts sidebar** (timings, best-for, etc. from the content meta). A **breadcrumb** shows Home / Amenities / (this amenity). Unknown slugs fall through to the 404 page. *Value:* sells the individual experience and supports SEO with dedicated pages.

3.5 Stays list /stays

Purpose: the accommodation catalogue and the top of the room-booking funnel.

Components: a **sort dropdown** and a grid of **StayCards**. Each card shows hero image, name, an optional **badge** (e.g. “Featured”), star **rating**, a short intro, price-per-night (formatted in Indian Rupees, e.g. “₹12,000”), and a link into the detail page. Seeded stays include **Wood House**, **Tower House Suite**, and **Kudil Cottage**. Fully data-driven with loading/empty/error states.

3.6 Stay detail /stays/:slug

Purpose: convince and convert on a specific room. It is the most commercially important public page.

COMPONENT	WHAT IT DOES
Gallery	Hero + multiple photos of the stay
Overview	Name, rating, capacity, beds (e.g. “1 Double Bed”), short intro, full description paragraphs
Amenities list	Per-stay amenity chips with icons
Accordions	Expandable sections (policies, details) — accessible with aria-expanded
Sticky Booking Widget	Date pickers + guests + price → launches the booking flow; stays visible as the user scrolls

3.7 Dining /dining

Purpose: present the food & beverage experience. **Components:** cuisine cards and signature spreads (data-driven, content type DINING) — **The Garden Grill, Saapadu Thali, Authentic Tamil, Live Grill & BBQ, Continental & More** — plus a chef’s note. *Value:* F&B is a major on-site revenue driver and a wedding differentiator.

3.8 Events /events

Purpose: sell the property as a celebrations venue. **Components:** image hero, **event-type cards** — **Weddings, Birthdays, Corporate, Get-togethers** — a gallery, and a steps band mirroring the home “01/02/03” process. A CTA routes to the enquiry form. *Value:* events are typically the highest-value bookings.

3.9 Offers / Packages /offers

Purpose: merchandise promotional packages (content type OFFER), each with an image, title, excerpt, price label, and CTA. *Value:* lets marketing run seasonal campaigns that staff manage themselves in the admin.

3.10 Reserve / Enquiry /reserve · /enquiry

Purpose: capture a lead when the guest is not ready to pay online (typical for weddings). **Form fields:** Name, Email, Phone, Occasion, Guests, Message. On submit it creates an **Enquiry** that appears in the admin leads inbox. **Validation:** required fields and format checks; a success confirmation and error handling. *Value:* no lead is lost to a forgotten inbox.

3.11 Sign in / Sign up /signin

Purpose: let guests create an account so their bookings are saved to a profile. **Fields:** Full name, Email, Phone, Password (sign-up); Email + Password (sign-in). A demo account is provided (guest@example.com / Guest@123). *Technical note:* issues the same secure JWT token pair as staff, but with the CUSTOMER role — so a customer can never reach the admin.

3.12 My Account /account

Purpose: the guest's self-service area. **Components:** "Your account" profile block and a "My bookings" list, matched to the signed-in email. **Empty state:** "You don't have any bookings yet." with a "Browse stays" button. *Value:* reduces "what's my booking?" calls to the front desk.

3.13 404 Not Found *

Purpose: gracefully handle mistyped or dead links with an on-brand page and a "Back home" action, instead of a broken browser error. Reached by any unknown URL, including unknown stay/amenity slugs.

04

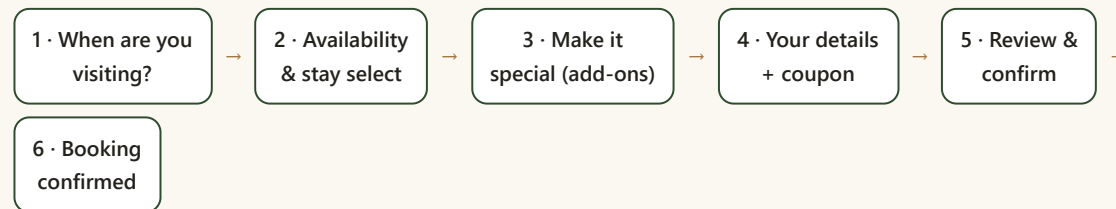
Booking & Payment Engine

The commercial heart of the platform — a multi-step guided flow with a real pricing engine and server-authoritative payments.

4 · The Booking & Payment Engine /book

This is the single most important workflow in the product. It converts a browser into a paying guest without any staff involvement.

4.1 The six steps (with exact on-screen labels)

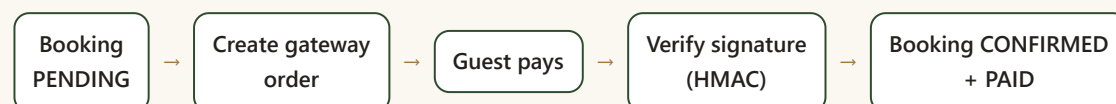


STEP	ON-SCREEN TEXT & CONTROLS	WHAT HAPPENS
1 · Dates	"Book your stay", "When are you visiting?", Check-in / Check-out pickers, "Select a stay..." dropdown	Captures dates & stay; a Stepper shows progress
2 · Availability	Per-night price quote, availability confirmation	Calls POST /availability/check — real price per night incl. rate rules & blocked dates
3 · Add-ons	"Make it special", "Optional extras — add what you'd like."	Optional up-sells added to the total
4 · Details	"Your details" — Full name, Email, Phone, Special requests; "Coupon code" field	Coupon validated live via POST /coupons/validate
5 · Review	"Review & confirm", line items, GST breakdown , "Total", "Test payment"/"Reserve"	Creates a PENDING booking, opens the payment gateway
6 · Confirmation	"Booking confirmed", booking code, "Browse more stays" / "Back home"	Shown only after the server confirms payment

4.2 The pricing engine (why the number is trustworthy)

The price is calculated on the **server**, never in the browser. It starts from the stay's base price-per-night, applies any matching **rate rule** for those dates (FIXED = set price, PERCENT = ±% of base, DELTA = ± flat amount, honouring minimum-stay and a priority order), rejects **blocked dates**, sums the nights, adds selected add-ons, subtracts a validated **coupon** (PERCENT with an optional cap, or FLAT, respecting minimum spend, usage limit and expiry), then adds **GST** per the Settings screen. The customer cannot tamper with any of this.

4.3 Server-authoritative payment (how real hotel engines work)



The browser **never** marks a booking as paid. The server confirms it only after a cryptographically verified callback (HMAC signature) or a Razorpay **webhook**. Two providers sit behind one abstraction: **Mock** (offline test checkout, the default — great for demos) and **Razorpay** (real orders, refunds, HMAC-verified webhooks). This design prevents the classic fraud where a user fakes a “paid” response.

4.4 Booking status state machine



Allowed moves are enforced on the server and mirrored in the admin UI: PENDING→CONFIRMED/CANCELLED; CONFIRMED→CHECKED_IN/CANCELLED; CHECKED_IN→CHECKED_OUT. Terminal states (CHECKED_OUT, CANCELLED) cannot move on. This prevents impossible transitions like checking out a cancelled booking.

Refunds are policy-aware. Cancel inside the free-cancellation window → full refund; outside it → the configured penalty is withheld. Issued from the booking drawer; the system cancels the booking and records a Payment row.

05

Admin Portal

The control room. Screen-by-screen documentation of every management tool, table, drawer, filter, button and permission.

5 • Admin Portal — Screen by Screen

5.0 Admin chrome (every admin page)

Sidebar — vertical navigation listing every admin section (Dashboard, Bookings, Availability, Stays, Amenities, Dining, Menu, Events, Offers, Coupons, Banners, Media, Reviews, Enquiries, Reports, Users, Settings, Email, Payments, Audit). Items the current role cannot access are hidden. **Topbar** — page context, signed-in user, and sign-out. **PageHeader** — title + primary action per screen. Shared UI kit: Badge, AdminButton, Drawer (slide-over panels), Field (labelled inputs), and DataState (loading/empty/error). A global **axios client** attaches the token and silently refreshes it once on a 401.

5.1 Admin Login /admin/login

Purpose: secure staff entry. **Components:** Email + Password fields, sign-in button, and **one-click “fill” buttons** for the three demo roles. **Validation:** wrong credentials show an error; success stores tokens and routes to the Dashboard. Roles & demo logins: Super Admin `admin@shraddhagarden.com/Admin@123`, Manager `manager@.../Manager@123`, Front Desk `frontdesk@.../Frontdesk@123`.

5.2 Dashboard /admin

Purpose: the at-a-glance health of the business. **Components:** **KPI cards** (totals such as revenue, bookings, enquiries), a **revenue chart** (6-month trend, Recharts) and a **bookings-by-status** breakdown. Data from GET `/dashboard/stats`. Loading/empty/error states throughout. *Value:* owners see performance without exporting anything.

5.3 Bookings /admin/bookings

Purpose: the operational core — manage every reservation. **Components:** a powerful **data table** (TanStack) with **search** (by name/code/email), a **status filter**, and **pagination**. Columns include code, guest, stay, dates, nights, guests, amount, status badge and payment badge. A row opens the **Booking Drawer**:

- **Detail view** of the guest, stay, dates and money.
- **Status buttons** that only offer legal next states (the state machine).
- **Payment actions** (mark paid, refund — policy-aware; Manager+).
- Notes and source (Website / Phone / Walk-in / OTA).

New Booking Drawer lets staff create a booking manually (walk-ins / phone) — pick stay, dates, guests; the server computes nights and amount.

5.4 Availability & Pricing /admin/availability (Manager+)

Purpose: control what can be booked and for how much. **Components:** an **availability checker**, a table of **rate rules** (name, stay or all-stays, date range, kind FIXED/PERCENT/DELTA, amount, min-stay, priority, active) with create/edit/delete, and a **blocked-dates** manager (date

range + reason). *Value*: seasonal pricing, weekend premiums and maintenance holds — the levers a revenue manager needs.

5.5 Stays /admin/stays

Purpose: manage the room inventory. **Components:** a **card grid** of stays and a create/edit **drawer form** (React Hook Form + Zod) covering name, slug, badge, price, rating, intro, description, hero image, gallery, amenities, capacity, beds, inventory, featured & published toggles. **Delete** is Super-Admin-only and is blocked if the stay has bookings (protects historical records). *Value*: the property's sellable catalogue is fully self-managed.

5.6 Content Manager — Amenities / Dining / Events / Offers (Manager+)

Purpose: one reusable CRUD screen powers four public sections. **Components:** a list of content items with published/featured flags and sort order, plus a **ContentForm** for title, slug, category, icon, excerpt, body paragraphs, hero image, gallery, tags, price label and a flexible **meta** block (what-to-expect, quick facts, timings, best-for). *Value:* marketing updates go live in seconds without a developer; one engine keeps four pages consistent.

5.7 Menu /admin/menu (Manager+)

Purpose: manage the food menu shown in dining materials. **Components:** menu **categories** (name, note, sort order, published) each holding **items** (name, price in rupees, veg/non-veg flag, availability). *Value:* keeps pricing and dishes current across breakfast/lunch/beverages/non-veg sections.

5.8 Coupons /admin/coupons (Manager+)

Purpose: run promotions. **Fields per coupon:** code, description, kind (PERCENT / FLAT), value, minimum spend, maximum discount cap, start/end dates, usage limit, used count, active. *Value:* measurable, controlled discounting with guard-rails against abuse (caps + limits + expiry).

5.9 Banners / Home content /admin/banners (Manager+)

Purpose: control the home page merchandising. **Three banner types:** ANNOUNCEMENT (the dismissible top strip), HERO (home hero slides), and PROMO (promotional banners, with a countdown from the end date). Each has title, subtitle, image, CTA label & link, active flag, sort order and scheduling. *Value:* time-boxed campaigns that switch themselves on/off by date.

5.10 Media /admin/media (Manager+)

Purpose: a central library of image URLs (with alt text and folder) reused across stays and content. *Value:* consistent imagery, one place to manage links. *Limitation (see audit):* currently URL-based, not direct file upload.

5.11 Reviews /admin/reviews (Front Desk+)

Purpose: a **moderation queue** for guest reviews (PENDING → APPROVED / REJECTED) with an optional public **reply**. Only approved reviews appear on the site. *Value:* protects brand reputation while showcasing genuine social proof.

5.12 Enquiries /admin/enquiries (Front Desk+)

Purpose: a lightweight CRM / leads inbox. **Components:** list of enquiries with status **NEW** → **CONTACTED** → **CLOSED**, an **assignee**, source, and a **convert-to-booking** action. *Value:* turns website enquiries into a trackable sales pipeline; nothing slips through the cracks.

5.13 Reports /admin/reports (Manager+)

Purpose: management reporting. **Components:** revenue, occupancy and booking-source breakdowns, with **CSV export** for finance/accounting. *Value:* board-ready numbers without a data analyst.

5.14 Users & Staff /admin/users

Purpose: manage the team. **Components:** user list with a role filter; **create staff and change roles** is Super-Admin-only. *Value:* onboard/offboard staff and enforce least-privilege access.

5.15 Settings /admin/settings (Manager+)

Purpose: global business configuration. **Fields:** GST / tax rate (feeds the booking price breakdown), contact information, and check-in / check-out times. **Value:** one place to keep legal/tax and operational parameters correct.

5.16 Email Settings /admin/email (Manager+)

Purpose: configure transactional email. The API includes a mailer, templates, a notify layer, and a **scheduler** that sends one-off reminders and review-requests (each booking records reminderSentAt and reviewRequestSentAt so nothing is sent twice). **Value:** automated guest communication — confirmations, pre-arrival reminders, post-stay review asks.

5.17 Payments /admin/payments

Purpose: configure how money is collected. **Fields:** provider (Mock / Razorpay), test/live mode, Key ID, Key Secret, Webhook Secret, enabled methods (card / UPI / net-banking / wallet), deposit %, and the cancellation policy. **Security:** secrets are editable by **Super Admin only** and are **never returned to the browser** — the public config exposes only the publishable Key ID. **Value:** go-live with real payments is a settings change, not a code change.

5.18 Audit Log /admin/audit (Super Admin only)

Purpose: accountability. **Components:** a chronological record of every admin mutation — actor, action, entity, entity id, timestamp. **Value:** answers “who changed this price / cancelled this booking / edited this user?” — essential for trust, disputes and (future) compliance.

Coverage note. Every admin screen shares the same three-state discipline (loading / empty / error), the same permission gate, and the same design language — the hallmark of a coherent, maintainable product rather than a collection of pages.

5.19 Admin permission matrix

CAPABILITY	FRONT DESK	MANAGER	SUPER ADMIN
View dashboard, bookings, enquiries, reviews	✓	✓	✓
Change booking status (check-in/out, cancel)	✓	✓	✓
Set payment status / issue refunds	—	✓	✓

Manage stays, availability, pricing, content, coupons, banners, media, reports, settings	—	✓	✓
Edit payment secrets / go live	—	—	✓
Create staff / change roles / delete stays	—	—	✓
View Audit Log	—	—	✓

6 • Shared UI Component & Text Catalogue

Every reusable building block and the meaning of the recurring text it carries.

6.1 Public components

COMPONENT	ROLE & TYPICAL TEXT
Header / Nav	Home · About · Amenities · Stays · Dining · Events · Offers + Reserve CTA
AnnouncementBar / MarqueeBanner	Admin-driven promo strip; dismissible (×)
Footer	Tagline, quick links, contact, copyright
Breadcrumb	"Home / Section / Item" trail on detail pages
StayCard / AmenityCard / DiningCard / TeamCard	Image + title + meta + link; badges & ratings
BookingWidget	Dates + guests + price → start booking (sticky on stay detail)
Gallery	Image lightbox/carousel on detail pages
Accordion	Expand/collapse (aria-expanded) for policies/details
Stepper	Progress indicator in the booking flow
StatsRow / StepsBand / SectionHeading / SectionEyebrow	Marketing layout primitives with italic "eyebrow" labels
Reveal / WordReveal / SiteAnimations	Scroll-reveal animations honouring reduced-motion
PublicState	Loading / Empty / Error renderer for data pages
Button / Pill / Icon	Primary/secondary actions, tags, iconography

6.2 Admin components

COMPONENT	ROLE
AdminLayout / Sidebar / Topbar / PageHeader	The admin shell & navigation
Drawer	Slide-over panels for detail/create/edit (bookings, stays, content)
Badge	Colour-coded status/payment indicators
DataState	Loading / empty / error for every data view
Field / AdminButton	Consistent labelled inputs & buttons
apiClient / tokenStore / queries	Auth-aware data layer (axios + token refresh + TanStack Query)

6.3 Recurring status vocabulary (what the badges mean)

DOMAIN	VALUES
Booking status	PENDING · CONFIRMED · CHECKED_IN · CHECKED_OUT · CANCELLED
Payment status	UNPAID · PAID · REFUNDED
Enquiry status	NEW · CONTACTED · CLOSED
Review status	PENDING · APPROVED · REJECTED
Rate rule kind	FIXED · PERCENT · DELTA
Coupon kind	PERCENT · FLAT
Banner type	ANNOUNCEMENT · HERO · PROMO
Booking source	Website · Phone · Walk-in · OTA

7 · Technical Architecture (Plain Language)

LAYER	WHAT IT IS (PLAIN)	HOW IT'S BUILT
Frontend	The screens guests & staff use	React 18 + Vite + TypeScript, Tailwind CSS, Framer Motion; admin code-split so heavy libraries never load for guests
Backend / API	The engine that enforces rules & holds data	Node.js + Express with clear route modules (auth, stays, bookings, payments, ...)
Authentication	Proving who you are	JWT access tokens (15-min) + rotating refresh tokens; bcrypt-hashed passwords
Authorization (RBAC)	Deciding what you may do	Roles + <code>requireRole</code> middleware on every write; UI hides what you can't use
Database	Where everything is stored	Prisma ORM over SQLite (16 models: User, Stay, Booking, Payment, Coupon, Content, Banner, Review, Media, RateRule, DateBlock, Enquiry, AuditLog, MenuCategory/Item, SiteSetting)
Payments	Collecting money safely	Provider abstraction (Mock / Razorpay), server-verified via HMAC signatures & webhooks
Email	Talking to guests automatically	SMTP mailer + templates + scheduler (reminders, review requests)
Validation	Rejecting bad input	Zod schemas on the API; React Hook Form + Zod on the forms
State / data-fetching	Keeping screens in sync	TanStack Query (caching, loading/error states, retries)
Security extras	Guard-rails	Secrets kept server-side; audit log; state-machine-guarded bookings; input serialization

7.1 Request lifecycle (a booking, end-to-end)

1. Guest submits dates & stay → React calls `POST /availability/check`.
2. Server prices the stay (base + rate rules – blocks) and answers.
3. Guest adds extras, details, coupon → React calls `POST /coupons/validate`, then `POST /bookings/public`.
4. Server creates a `PENDING` booking & a gateway order (`POST /payments/order`).
5. Guest pays → gateway calls back → `POST /payments/verify` (HMAC) and/or webhook.
6. Server flips the booking to `CONFIRMED` + `PAID`, records a `Payment` row, triggers a confirmation email, writes nothing the browser could forge.

Honest note on deployment & ops. The code ships with an nginx config and deploy/update scripts, but there is **no evidence of automated CI/CD, centralized**

logging, metrics/monitoring, or infrastructure-as-code in the repo. These are addressed as production-readiness items in §9–§12 rather than assumed to exist.

8 • Functional Analysis — Input / Process / Output

FEATURE	INPUT	PROCESS	OUTPUT	KEY EDGE / FAILURE CASES
Availability check	Stay, check-in, check-out	Apply rate rules, reject blocks, sum nights	Per-night & total quote; available/not	Check-out $\leq$ check-in; blocked range; past dates
Coupon validate	Code, cart amount	Check active, dates, min-spend, usage limit, cap	Discount amount or rejection reason	Expired, over-limit, below min-spend, unknown code
Create booking	Dates, stay, guest details	Recompute price server-side; create PENDING	Booking code + order	Race on inventory; invalid email/phone
Payment verify	Gateway signature/payload	HMAC verify; idempotent confirm	CONFIRMED + PAID or failure	Forged signature; duplicate webhook; timeout
Status transition	Target status	State-machine guard	Updated booking	Illegal transition rejected
Refund	Booking, policy window	Compute penalty; call gateway	REFUNDED + CANCELLED + Payment row	Already refunded; outside gateway window
Enquiry submit	Name, email, phone, message...	Validate; store as NEW	Lead in admin inbox	Spam/bot input (no captcha — see audit)
Content/Stay CRUD	Form fields	Zod validate; RBAC check; persist	Updated public page	Duplicate slug; unauthorized role
Login	Email + password	bcrypt compare; issue tokens	Access + refresh JWT	Wrong creds; inactive user; token expiry→refresh

Expected behaviour, stated plainly. The system consistently prefers *fail-safe* outcomes: money is only ever confirmed by the server; impossible booking states are refused; unauthorized actions are blocked before they touch data; and every data screen degrades gracefully to a readable message rather than a blank crash.

09

Product Audit & Risk

A brutally honest assessment: what is missing, what is weak, and what must be true before large-scale production.

9 • Product Audit — Gaps & Missing Features

The product is unusually complete for its stage. The following are genuine gaps, grouped by theme, each with an impact rating.

INFRASTRUCTURE & SCALE

GAP	WHY IT MATTERS	IMPACT
SQLite database	Excellent for one property/dev, but single-file and limited for high concurrency or multi-property scale; migrate to PostgreSQL/MySQL for production	High
No automated tests	No unit/integration/e2e suite found; regressions can reach production undetected	High
No CI/CD pipeline	Deploys are manual scripts; higher human-error risk	Med
No monitoring/alerting/centralized logs	Outages/errors discovered by customers, not dashboards	High
Media is URL-only (no file upload/storage/CDN)	Staff must host images elsewhere; no drag-drop upload	Med
No backup/restore strategy documented	Data loss risk for bookings/payments	High

FUNCTIONAL / WORKFLOW

GAP	WHY IT MATTERS	IMPACT
Inventory / double-booking concurrency	Two guests booking the last unit simultaneously needs a hard lock/transaction; verify robustness under load	High
No calendar/channel-manager sync (OTA)	If the resort also sells on OTAs, availability can drift	Med
No spam protection on public forms (captcha/rate-limit)	Enquiry & review forms open to bots	Med
No booking modification by guest (self-service reschedule)	Changes require calling the front desk	Low
No multi-currency / i18n / multi-language	INR & English only; limits international guests	Low
No invoice/GST PDF generation for guests	Common expectation for corporate bookings	Med

COMPLIANCE, ANALYTICS & TRUST

GAP	WHY IT MATTERS	IMPACT
No cookie consent / privacy & T&C pages	Legal requirement (DPDP Act, India) before public launch	High
No web analytics (GA4/traffic funnels)	Can't measure conversion or marketing ROI	Med
Audit log covers admin only, not customer actions	Partial trail; fine for now, expand later	Low
No rate-limiting / WAF on the API	Brute-force & abuse exposure	Med

10 • Negative Analysis (Brutally Honest)

Read this section as a friend, not a salesperson. Everything below is fixable, and none of it is unusual for a product at this stage — but a client deserves to hear it before signing.

Technical risks

- **SQLite is the biggest single technical risk.** It is superb for development and a single small property, but it is a file-based database. Concurrency, backups and multi-server scaling are limited. This must move to PostgreSQL before serious volume.
- **No automated test coverage.** Every future change is a manual-QA gamble. For a system that *touches money*, this is the most important thing to add.
- **Operational blind spot.** Without logging/monitoring, the first sign of a payment or email failure may be an angry guest. Add error tracking (e.g. Sentry) and uptime monitoring day one.
- **Concurrency/idempotency.** The payment verify path must be provably idempotent (a webhook can fire twice). This should be load-tested and confirmed, not assumed.

Business & UX risks

- **Content dependency.** The product's premium feel relies on high-quality photography and copy. Placeholder/Unsplash imagery must be replaced with real property shots before launch or the brand suffers.
- **No self-service booking changes.** Guests who want to move a date must phone in — a friction point competitors solve.
- **Legal pages missing.** Privacy policy, terms, cancellation/refund policy and cookie consent are prerequisites, not nice-to-haves, in India.
- **Single-property assumption.** The data model is single-property. Expanding to multiple resorts is a real project, not a config toggle.

Likely future client complaints (anticipate them)

COMPLAINT	ROOT CAUSE	PRE-EMPTIVE ANSWER
"It went down and no one told us"	No monitoring	Add Sentry + uptime alerts in hardening sprint
"A change broke something else"	No tests	Introduce a test suite before feature work
"I can't upload photos directly"	URL-only media	Add upload+storage (S3/Cloudinary) in v1.1
"Guests want an invoice PDF"	Not built	Add GST invoice generation in v1.1
"We got spam enquiries"	No captcha	Add captcha + rate-limit in hardening sprint

Balancing truth. Despite the above, the *architecture* is sound and the *feature surface* is broad and coherent. These are hardening items, not rewrites. That distinction is what makes this a strong investment rather than a risky one.

11 • Risk Matrix

RISK	LIKELIHOOD	IMPACT	SEVERITY	MITIGATION
Database can't scale (SQLite)	High	High	Critical	Migrate to PostgreSQL before volume launch
Undetected regressions (no tests)	High	High	Critical	Add unit/integration/e2e suite + CI
Silent outage / payment failure	Med	High	High	Sentry + uptime + log aggregation
Data loss (no backups)	Med	High	High	Automated daily backups + tested restore
Double-booking under load	Med	High	High	DB transaction/lock on inventory + load test
Legal non-compliance at launch	High	Med	Med	Add privacy/T&C/cookie consent pages
Form spam / API abuse	Med	Med	Med	Captcha + rate-limiting + WAF
Brand harm from placeholder media	Med	Med	Med	Professional photo shoot before launch
Key-person/vendor lock (Razorpay only)	Low	Med	Low	Provider abstraction already exists — add a 2nd gateway later
Guest friction (no self-reschedule)	Med	Low	Low	Add self-service changes in v1.5

Reading the matrix: the two “Critical” rows (database & testing) plus the four “High” rows constitute the **hardening sprint** — roughly 4–6 weeks of focused work — that turns a strong build into a launch-ready production system.

12 · Production Readiness Scorecard

Scores out of 10 per dimension. “Business readiness” = how usable it is to run the business today; “Backend readiness” includes ops maturity.

MODULE	UI	UX	BACKEND	BUSINESS	PERF.	SECURITY	SCALE	TESTING	MAINT.	DOCS	DEPLOY
Public website	9	9	8	9	8	8	6	2	8	8	6
Booking & payments	8	8	9	9	7	9	6	2	8	8	6
Admin portal	9	8	8	9	8	8	6	2	8	8	6
Availability & pricing	8	7	8	8	7	8	6	2	8	7	6
Content / CMS	8	8	8	9	8	8	7	2	8	7	6
Auth & RBAC	8	8	9	8	8	9	7	2	8	8	6
Reporting & audit	7	7	8	8	7	8	6	2	8	7	6
Infrastructure / ops	—	—	5	6	6	6	4	2	6	6	4

8.4

avg. feature quality

8.6

security design

2.0

test coverage

5.0

ops maturity

Interpretation. Feature & design maturity is genuinely high (8–9). The scorecard’s low columns — **testing (2)** and **ops/scale (4–6)** — are exactly the hardening backlog. This is the profile of a *strong MVP*, not a fragile prototype.

13 · Future Roadmap & Timeline

RELEASE	THEME	HEADLINE ITEMS
MVP (now)	Direct-booking platform	Public site, admin portal, booking + payments, RBAC, CMS, coupons, reviews, reports, audit
Hardening sprint	Production-ready	PostgreSQL migration, automated tests + CI/CD, monitoring/logging, backups, captcha/rate-limit, legal pages
v1.1	Operator delight	Direct image upload + CDN, GST invoice PDFs, web analytics (GA4), email template editor
v1.5	Guest self-service	Self-reschedule/cancel, saved guests, loyalty points, WhatsApp notifications, wishlists
v2.0	Multi-property & channels	Multi-resort data model, OTA channel-manager sync, dynamic yield pricing, housekeeping module
Enterprise Edition	Scale & compliance	SSO/SAML, granular custom roles, data residency, SLA monitoring, white-label, API for partners

13.1 Themed roadmaps

AI

Chat concierge, AI enquiry auto-reply/triage, review sentiment summaries, demand-forecast pricing suggestions, AI copy for content.

AUTOMATION

Auto-confirm/reminders (exists) → auto-refund workflows, no-show handling, dynamic coupon issuing, housekeeping task automation.

ANALYTICS

Conversion funnels, RevPAR/ADR/occupancy dashboards, cohort & source ROI, forecast vs actual.

SECURITY

Rate-limiting/WAF, 2FA for admins, secret rotation, pen-test, PCI-scope review, DPDP compliance.

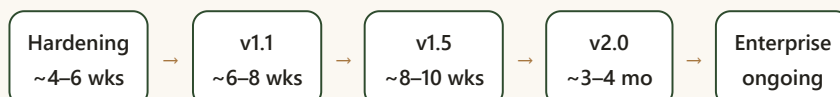
SCALABILITY

Postgres + read replicas, Redis cache, CDN, containerisation, horizontal API scaling, queue for emails/webhooks.

MOBILE & API

PWA/installable app, native guest app, public partner API, webhooks out, OTA integrations.

13.2 Indicative timeline



14 • Business Benefits & ROI

BENEFIT	WHAT IT MEANS FOR THE RESORT
Commission savings (ROI)	Every direct booking avoids the 15–25% OTA commission. On ₹50,000/day of direct bookings, ~₹7,500–₹12,500/day stays with the resort.
Cost savings	Replaces several subscriptions (website builder, booking engine, CRM, CMS) with one owned platform.
Productivity	Staff self-manage content, pricing and bookings; automated emails cut manual follow-up.
Security	Server-authoritative payments, hashed passwords, RBAC and audit logging reduce fraud & internal error.
Compliance readiness	Audit trail + settings-driven GST provide the backbone for tax & governance (with legal pages added).
Operational efficiency	One dashboard for bookings, leads, availability and reports — no spreadsheet reconciliation.
Business continuity	Central data with (planned) backups beats knowledge trapped in chats and inboxes.
Competitive advantage	A premium, mobile-first direct channel rivals larger chains; owned guest data enables marketing.
Future growth	Architecture already supports multi-provider payments and content-driven pages; multi-property is a clear next step.

The ROI story in one line: the platform pays for itself by converting OTA-commissioned bookings into direct, commission-free bookings — while giving the owner data and control they never had before.

15

100 Client Questions

Anticipated questions with professional answers, objection-handling and follow-ups — your Q&A shield for the meeting.

15 • 100 Client Questions & Answers

Answers blend business + simple-technical framing. Handle objections by acknowledging, reframing to value, and pointing to the roadmap.

QUESTION	PROFESSIONAL ANSWER
1. Is the website live?	It runs today via the included host scripts; production launch follows the hardening sprint (\$13).
2. Can guests really pay online?	Yes — via Razorpay (cards/UPI/net-banking/wallet), confirmed server-side. A Mock mode demos it with no real money.
3. Do we lose OTA commissions?	Direct bookings avoid the 15–25% OTA cut entirely.
4. Is my guests' payment data safe?	Card data is handled by the gateway, not stored by us; we verify payment cryptographically (HMAC).
5. Can staff change prices themselves?	Yes — seasonal/dynamic rate rules in Availability & Pricing, no developer needed.
6. Who can access what?	Four roles (Super Admin, Manager, Front Desk, Customer) with enforced permissions.
7. Can I see who changed something?	Yes — the Audit Log records every admin change (Super Admin only).
8. Does it work on phones?	Fully responsive across phone/tablet/desktop.
9. Can we run offers/discounts?	Yes — coupons (percent/flat, caps, limits, expiry) and scheduled promo banners.
10. How do enquiries reach us?	They land in a leads inbox with status, assignee and convert-to-booking.
11. Can we manage the menu?	Yes — categories & items with prices and veg flags.
12. Can we edit amenities/dining/events/offers?	Yes — one CMS engine powers all four public sections.
13. Can we moderate reviews?	Yes — approve/reject/reply; only approved reviews show publicly.
14. Can we export data?	Reports export to CSV for finance/accounting.
15. What reports do we get?	Revenue, occupancy and source breakdowns, plus a KPI dashboard with charts.
16. Can we take walk-in/phone bookings?	Yes — staff create bookings manually; source is tracked.
17. What stops double-booking?	Inventory + blocked dates; concurrency hardening is on the roadmap and will be load-tested.
18. Can we block dates for maintenance?	Yes — blocked-date ranges with a reason.

19. How are refunds handled?	Policy-aware from the booking drawer: full inside the free window, penalty outside it.
20. Do guests get confirmation emails?	Yes — automated confirmations, reminders and review requests (sent once each).
21. Can guests see their bookings?	Yes — a My Account area lists bookings by email.
22. Do guests need an account to book?	They can book as a guest; accounts add history & convenience.
23. Is it fast?	Modern build, code-splitting and caching; admin libraries never load for guests.
24. Is it accessible?	Semantic landmarks, alt text, focus rings, reduced-motion support.
25. Who owns the data?	You do — it's your platform and your database.
26. Can we add more room types?	Yes — Stays are fully self-managed with a rich form.
27. Can we add photos?	Yes, via image URLs today; direct upload is a v1.1 item.
28. What tech is it built on?	React front end, Node/Express API, Prisma database toolkit — mainstream, hireable stack.
29. Will we be locked to one developer?	No — standard tech, documented, with a clear structure.
30. Can it scale to more guests?	Yes after moving to PostgreSQL and adding caching/CDN (roadmap).
31. Can it handle multiple resorts?	Not yet — multi-property is a defined v2.0 project.
32. Is there a mobile app?	Not yet; a PWA/native app is on the roadmap. The site is already mobile-first.
33. What happens if the payment fails?	The booking stays PENDING and is never marked paid; the guest can retry.
34. Can a hacker fake a payment?	No — only the server confirms payment via a verified signature/webhook.
35. Are passwords stored safely?	Yes — hashed with bcrypt, never in plain text.
36. How long do logins last?	15-minute access tokens auto-refresh silently; users aren't logged out mid-task.
37. Can we add staff?	Yes — Super Admin creates staff and assigns roles.
38. Can Front Desk change prices?	No — pricing is Manager+ only, by design.
39. What if we type a wrong URL?	A friendly on-brand 404 page with a way back home.
40. Can we change tax/GST?	Yes — in Settings; it feeds the booking price breakdown.

QUESTION	PROFESSIONAL ANSWER
41. Can we set check-in/out times?	Yes — in Settings.
42. Can we schedule a promo to start later?	Yes — banners & coupons have start/end dates.
43. Is there a countdown for offers?	Yes — PROMO banners show a countdown from the end date.
44. Can we feature certain stays?	Yes — a “featured” flag surfaces them.
45. Can we hide a stay temporarily?	Yes — a “published” toggle.
46. Can we see revenue trends?	Yes — a 6-month revenue chart on the Dashboard.
47. Can we see bookings by status?	Yes — a status breakdown on the Dashboard.
48. How do we find a booking fast?	Search by name/code/email, filter by status, paginated table.
49. Can we add booking notes?	Yes — notes and source on each booking.
50. Can we assign an enquiry to someone?	Yes — assignee field on each lead.
51. Is the design consistent?	Yes — one design system across public & admin.
52. Can we change the branding?	Yes — colours/typography are centralised tokens.
53. Is it SEO-friendly?	Dedicated pages per stay/amenity help; meta/analytics are roadmap items.
54. Does it support multiple languages?	Not yet — i18n is on the roadmap; English/INR today.
55. Can international guests pay?	Via Razorpay’s supported methods; multi-currency is future scope.
56. What about GST invoices?	GST is computed now; guest-facing invoice PDFs are a v1.1 item.
57. Is there spam protection on forms?	Not yet — captcha + rate-limit are in the hardening sprint.
58. Is there a privacy policy?	Not yet — legal pages are a launch prerequisite we’ll add.

59. Are there automated tests?	Not yet — the top hardening priority given payments are involved.
60. What if the server goes down?	We'll add uptime monitoring + alerts so we know before guests do.
61. Are there backups?	To be implemented — automated daily backups with tested restore.
62. Which database is used?	SQLite today (great for launch); PostgreSQL for scale (planned).
63. Why not Postgres now?	SQLite ships zero-setup and is perfect to validate the product; migration is straightforward.
64. How do we deploy updates?	Included nginx config + deploy scripts today; CI/CD is planned.
65. Can we integrate with OTAs?	Channel-manager sync is a v2.0 item; the architecture anticipates it.
66. Can guests reschedule themselves?	Not yet — self-service changes are v1.5; staff can do it now.
67. Can we do loyalty/points?	Planned for v1.5.
68. WhatsApp notifications?	Planned; email automation exists today.
69. Can we A/B test offers?	Not built-in; analytics + experiments are roadmap.
70. How secure is admin access?	Gated by login + RBAC; 2FA is a planned security enhancement.
71. Can we limit refunds by policy?	Yes — a configurable cancellation policy drives refunds.
72. Can we take partial deposits?	Yes — a deposit % is configurable in Payments.
73. Which payment methods?	Card, UPI, net-banking, wallet — toggle per method.
74. Can we switch payment provider?	Yes — a provider abstraction; adding another gateway is low-risk.
75. Are payment secrets exposed?	Never to the browser — only the public Key ID is; secrets are Super-Admin-only.
76. Can we test before going live?	Yes — Mock mode + Razorpay test keys exercise the full flow safely.
77. How do we train staff?	The admin mirrors familiar dashboards; we'll provide a short guide + demo logins.
78. Can we restrict deletions?	Yes — deleting stays is Super-Admin-only and blocked if bookings exist.
79. What if two staff edit at once?	Last-write-wins today; optimistic-locking is a maintainability enhancement.
80. Is the code maintainable?	Yes — typed (TypeScript), modular routes, shared components, documented README.

QUESTION	PROFESSIONAL ANSWER
81. What's the total cost of ownership?	Hosting + gateway fees + occasional dev; far below stacked SaaS subscriptions + OTA commissions.
82. How fast can we launch?	Demo today; production after a ~4–6 week hardening sprint.
83. What's the biggest risk?	Honestly, the database choice and lack of tests — both on the immediate plan.
84. Why should we trust the numbers on screen?	All money maths is server-side; the browser can't alter prices.
85. Can we see occupancy?	Yes — in Reports, with source breakdowns.
86. Can we brand emails?	Templates exist; a visual editor is a v1.1 item.
87. Does it handle time zones/dates well?	Dates are validated server-side; we'll confirm timezone handling in hardening.
88. Can we add custom roles?	Not yet — granular custom roles are an Enterprise-edition feature.
89. Is there an activity history per booking?	Status changes flow through a guarded state machine; a full per-entity timeline is an enhancement.
90. Can we see failed payments?	Payment records capture status (created/paid/failed/refunded).
91. What about accessibility compliance?	Strong foundations; a formal WCAG audit is recommended pre-launch.
92. Can we localise currency formatting?	INR formatting is built in; other locales are future scope.
93. Is there a sitemap/robots for SEO?	To be added with the analytics/SEO roadmap item.
94. Can we gate content by login?	Customer auth exists; member-only content is a small extension.
95. How do we handle no-shows?	Manually today (cancel); automation is roadmap.
96. Can we integrate accounting software?	CSV export today; direct integrations are future scope.
97. Is the platform ours to resell/white-label?	White-label is an Enterprise-edition direction.
98. What ongoing support is needed?	Light — content is self-managed; dev needed for features & upgrades.
99. What happens to bookings if we change a stay?	Historical bookings are protected; stays with bookings can't be deleted.

100. Why choose this over an off-the-shelf builder?

You own the data, avoid commissions, and can extend infinitely — an off-the-shelf tool can't match owned, tailored software.

Objection-handling cheat sheet

OBJECTION	REFRAME
"It's not 100% production-ready."	Correct — and we've told you exactly what the 4–6 week hardening sprint covers. Transparency now beats surprises later.
"No tests worries me."	Agreed it's the #1 priority; it's the first item on the plan precisely because money is involved.
"SQLite sounds small."	It's ideal to launch fast and cheap; migrating to PostgreSQL is a well-trodden, low-risk step.
"Can we afford it?"	The OTA commissions you avoid typically cover the build; this is a cost-saver, not just a cost.

16 • Presentation Scripts

16.1 Ten-minute script (the elevator pitch)

1. **(0–1 min) Open:** “Shraddha Garden now has its own online front door and its own control room.” State the tagline.
2. **(1–3) The guest story:** Walk the Home → Stays → Stay detail → Book flow. Emphasise real online payment and instant confirmation.
3. **(3–6) The control room:** Show the Dashboard, Bookings table + drawer, Availability & Pricing, and Content CMS. “Your team runs everything from here.”
4. **(6–8) Why it matters:** Commission-free direct bookings, owned data, audit trail, roles.
5. **(8–10) Honesty + close:** “It’s a strong MVP; a short hardening sprint takes it to production. Here’s the plan.” Invite questions.

16.2 Thirty-minute script

1. **(0–3) Context & problem** (§1): OTA commissions, lost enquiries, spreadsheet pricing.
2. **(3–10) Guest experience** (§3–4): full public tour + booking & payment engine; explain server-authoritative payments in plain terms.
3. **(10–20) Admin deep-dive** (§5): dashboard, bookings state machine, availability/pricing, CMS, coupons, banners, reviews, enquiries, reports, users, payments, audit — highlight the permission matrix.
4. **(20–24) Architecture & security** (§7): RBAC, hashed passwords, HMAC payments, audit log — confidence without jargon.
5. **(24–28) Audit & roadmap** (§9–13): what’s missing, the hardening sprint, and where it grows (multi-property, AI, mobile).
6. **(28–30) ROI & close** (§14): the commission-savings story; agree next steps.

16.3 One-hour detailed walkthrough (agenda)

TIME	SEGMENT	ANTICIPATED QUESTIONS TO PRE-EMPT
0–5	Executive summary (§0)	“What is this, in one sentence?”
5–15	Full public site tour (§3)	“Does it work on phones?” “Can we edit content?”
15–25	Booking + payments live demo (§4)	“Is payment safe?” “What if it fails?”
25–40	Admin portal, screen by screen (§5)	“Who can do what?” “Can we see who changed things?”
40–48	Architecture & security (§7–8)	“Will we be locked in?” “Can it scale?”
48–56	Audit, risks, readiness, roadmap (§9–13)	“Is it production-ready?” “What’s missing?”
56–60	ROI, Q&A, next steps (§14–15)	“What does it cost?” “When can we launch?”

Presenter tip. Lead every screen with the *business* value, then reveal the *mechanism*. When asked a hard technical question, answer in one plain sentence, then offer “I can go deeper if useful.” Confidence comes from control, not from detail.

17 · Conclusion & Recommendation

Shraddha Garden Resort is a genuinely impressive, coherent product: a premium guest website, a comprehensive admin portal, and a secure, well-designed API — unified by one design system and one set of business rules.

What is strong

- Broad, coherent feature surface: booking, payments, pricing, CMS, coupons, banners, reviews, enquiries, reports, users, settings, audit.
- Sound security posture: RBAC, hashed passwords, server-authoritative HMAC-verified payments, secrets kept server-side, full admin audit trail.
- Thoughtful UX: consistent loading/empty/error states, accessibility foundations, responsive design, guided booking flow.
- Extensible architecture: provider-abstracted payments, content-driven pages, code-split admin.

What must happen before large-scale production

1. Migrate SQLite → PostgreSQL.
2. Add automated tests + CI/CD.
3. Add monitoring, logging, alerting and backups.
4. Add captcha/rate-limiting and legal pages (privacy, T&C, cookie consent).
5. Load-test booking concurrency & payment idempotency.
6. Replace placeholder media with professional property photography.

Recommendation. Proceed. Position the current build as a **strong MVP**, commission a focused **4–6 week hardening sprint**, then launch. The commercial case — commission-free direct bookings on an owned, extensible platform — is compelling, and the technical foundation is genuinely good.

Shraddha Garden Resort — Product & Technical Documentation, v1.0 · 09 July 2026

Prepared for client presentation. This dossier documents the product as implemented in the codebase; items marked “planned/roadmap” are not yet built. © 2024 Shraddha Garden Resort. Unforgettable Memories.